



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus
Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200
Tel Nos.: (+63 43) 425-0139 local 2222 / 2223

E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Informatics and Computing Sciences

PROJECT PROPOSAL
Computer Science Department
2nd Semester AY 2025-2026

Course Code	CC 102
Course Description	Advanced Computer Programming
Section	CS - 1202
Date	
Time	

SR-Code	25-04677
Name of Student	Soreta, Anthony S.

Weight	5% of Final Grade – Project Proposal 5% of Final Grade – Presentation
ILO	ILO1: Design a Python program that meets a given specification using appropriate data structures, control structures, and standard libraries. ILO4: Present orally the design, algorithmic logic, and code structure of a Python program in a clear, well-organized, and technically accurate manner using appropriate computing terminology.
SO	SO1 Discipline Knowledge SO3 Communication
Bloom's Level	Create, Understand

PART	SECTIONS	POINTS	
I	PROBLEM ANALYSIS	10	
II	SYSTEM OBJECTIVES & REQUIREMENTS	10	
III	PROPOSED DATA STRUCTURES	10	
IV	CONTROL STRUCTURES AND PROGRAM FLOW	10	
V	HIGH-LEVEL SOLUTION DESIGN	10	
	TOTAL	50	
VI	TECHNICAL EXPLANATION	5	
	ORAL DELIVERY	5	
	TOTAL	10	

ILOs	SECTIONS	POINTS
1	I, II, III, IV, V	50
4	VI	10



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2222 / 2223

E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Informatics and Computing Sciences

GUITAR TUNER SYSTEM

I. Problem Description and Analysis

General Problem Description

Many beginner guitar players experience difficulty tuning their instruments accurately. Manual tuning methods rely heavily on the user's hearing ability and musical experience, which may lead to incorrect tuning and poor sound quality. While digital tuners exist, not all users have access to dedicated tuning devices.

Specific Sub-Problems

From the general problem, the following specific sub-problems are identified:

1. Difficulty in Identifying Correct Pitch

Beginners often struggle to determine whether a string is tuned to the correct pitch.

2. Lack of Accessible Tuning Tools

Not all guitar players have access to digital tuners or mobile applications.

3. Inconsistent Manual Tuning

Manual tuning may produce inconsistent results due to variations in hearing and interpretation.

Intended System Users

1. Beginner guitar players who need assistance in tuning their instruments
2. Students learning guitar who require a simple and accessible tuning tool
3. Musicians who want a basic tuner implemented through a computer program

Suitability for a Python-Based Solution

The problem is suitable for a Python-based solution because Python provides libraries capable of processing audio input and analyzing sound frequencies. Python's built-in data structures and control structures can be used to store standard guitar frequencies, compare detected sound signals, and determine whether the string is flat, sharp, or correctly tuned. Additionally, Python supports modular programming, making it suitable for implementing different functions for sound detection, data processing, and result display.

II. System Objectives and Requirements

Main Objective

To develop a Python-based guitar tuner system capable of detecting sound frequency and identifying whether a guitar string is correctly tuned.

Specifically, it aims to:

1. Detect the frequency of a sound produced by a guitar string using microphone input.
2. Compare the detected frequency with standard guitar tuning frequencies.
3. Display whether the string is **flat, sharp, or in tune**.

Functional Requirements

The proposed system shall:

1. Capture sound input from the computer or smartphone microphone.
2. Analyze the frequency of the detected sound signal.
3. Identify the closest guitar string based on standard tuning frequencies.
4. Display the tuning status (Flat, Sharp, In Tune).

Expected Program Inputs and Outputs

Inputs: Audio signal from microphone, detected frequency value, user command to start or stop tuning.



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University

Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2222 / 2223

E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Informatics and Computing Sciences

Outputs: Detected frequency (Hz), identified guitar string (E, A, D, G, B, E), tuning status (Flat, Sharp, In Tune).

III. Proposed Data Structures

The system will utilize the following Python data structures:

Dictionary: A dictionary will be used to store the standard guitar tuning frequencies. Each guitar string will serve as a key, while its corresponding frequency value (in Hertz) will serve as the value.

List: A list will be used to temporarily store detected frequency readings captured from the microphone. Multiple readings may be collected to compute an average frequency value, which helps improve the accuracy of pitch detection.

Justification

Dictionaries are appropriate for efficient retrieval of standard guitar frequencies using string identifiers as keys. Lists allow storage of multiple detected frequency readings for averaging and improved accuracy. These structures support organized and reliable frequency comparison in the tuning process.

IV. Control Structures and Program Flow

Use of Control Structures

- **Conditional statements** will compare the detected frequency with standard guitar tuning frequencies to determine whether the pitch is **Low, High, or In Tune**.
- **Loops** will be used to continuously capture and analyze sound input from the microphone while the program is running.
- **Recursive functions** are not required since frequency comparison and string detection can be handled using loops and built-in Python functions.

Program Flow Description

1. The program starts and prompts the user to begin tuning.
2. The system captures sound input from the microphone.
3. The program analyzes the frequency of the sound.
4. The detected frequency is compared with standard guitar tuning frequencies.
5. The closest guitar string is identified.
6. The program determines if the string is flat, sharp, or in tune.
7. The result is displayed to the user.

Key Decision Points

- Determining whether sound input is detected
- Selecting the closest guitar string
- Determining tuning status based on frequency difference

V. High-Level Solution Design (Program Outline)

Proposed Program Structure

- `display_menu()` – Displays program instructions and options for the user.
- `capture_audio()` – Records sound input through the system microphone.
- `detect_frequency()` – Analyzes the sound signal and calculates its frequency.
- `identify_string()` – Compares detected frequency with standard guitar tuning values and identifies the closest match.
- `checking_tuning()` – Determines whether the string is flat, sharp, or in tune.
- `display_result()` – Displays the detected string, frequency value, and tuning status.



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2222 / 2223

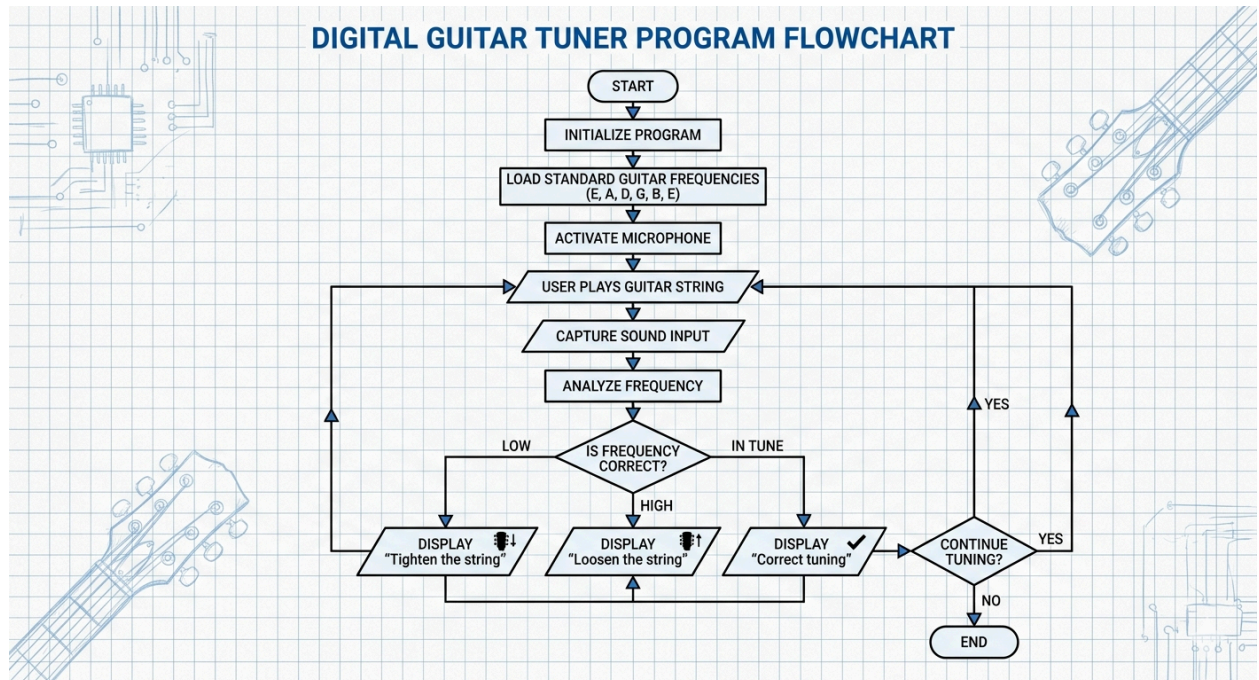
E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Informatics and Computing Sciences

This high-level design outlines the responsibilities of each function and supports modular program development.

Flowchart

A flowchart illustrating the overall program flow, decision points, and function calls will be provided.





Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2222 / 2223

E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Informatics and Computing Sciences

RUBRICS for PROJECT PROPOSAL

Criteria	10 – Excellent	8 – Very Good	6 – Satisfactory	4 – Needs Improvement	1 – Poor
Problem Description and Analysis	Clearly defines the general problem; identifies at least three well-articulated sub-problems; clearly identifies intended users; provides a strong and relevant justification for a Python-based solution, including its suitability for modular and automated processing	General problem is clear; three sub-problems identified with minor gaps; users and Python suitability explained adequately	Problem and sub-problems are present but lack clarity or depth; justification is weak or general	Problem description is vague; fewer than three sub-problems; limited discussion of users or Python suitability	Problem is unclear or missing; sub-problems not identified
System Objectives and Requirements	Primary objective is clearly derived from the problem; specific objectives directly map to sub-problems; at least three clear functional requirements are identified; inputs and outputs are clearly described and aligned with modular program design	Objectives and requirements are mostly aligned; minor gaps in mapping or clarity	Objectives and requirements are partially aligned; inputs and outputs are unclear or incomplete	Objectives or requirements are weakly stated and poorly aligned	Objectives and requirements are missing or incorrect
Proposed Data Structures	Appropriate data structures are clearly identified; stored data is well described; strong justification is provided for each selection, including how the data structures support system objectives and are used across functions or modules	Data structures are appropriate with adequate descriptions; justification is present but limited	Data structures are somewhat appropriate; descriptions or justification lack clarity	Data structures are poorly chosen or weakly explained	Data structures are missing or inappropriate
Control Structures, Procedural Abstraction, and Program Flow	Clearly explains the use of conditionals, loops, and recursive logic where applicable; identifies appropriate functions and their responsibilities; program flow from input to output is logical; key decision points are clearly identified	Control structures and program flow are explained with minor gaps; basic use of functions is evident	Explanation of control structures or program flow is limited; procedural abstraction or recursion is mentioned but unclear	Program flow is poorly described; limited or incorrect use of functions or control structures	Control structures, procedural abstraction, and flow are missing or incorrect
High-Level Solution Design (Flowchart / Program Outline)	High-level design is clear, complete, and consistent with stated objectives, requirements, data structures, and control structures; includes a well-organized program outline showing function responsibilities and modular structure	Design is clear but missing minor details in structure or modular organization	Design is partially complete or loosely connected to objectives and structure	Design is poorly structured or difficult to follow	No meaningful high-level design is provided



Republic of the Philippines
BATANGAS STATE UNIVERSITY
The National Engineering University
Alangilan Campus

Golden Country Homes, Alangilan Batangas City, Batangas, Philippines 4200

Tel Nos.: (+63 43) 425-0139 local 2222 / 2223

E-mail Address: cics.alangilan@g.batstate-u.edu.ph | Website Address: <http://www.batstate-u.edu.ph>

College of Informatics and Computing Sciences

RUBRICS for ORAL PRESENTATION

Criteria	5 – Excellent	4 – Very Good	3 – Satisfactory	2 – Needs Improvement	1 – Poor
Technical Explanation of Design and Logic (Pitch Content)	Delivers a clear, concise, and persuasive 3-minute pitch in English that accurately explains the program design, algorithmic logic, and code structure; key technical points are prioritized and communicated effectively within the time limit	Explains design and logic mostly in English with minor gaps or inaccuracies; pitch slightly under/over time (± 30 seconds) but key ideas remain clear	Provides a basic explanation of design and logic; partially delivered in English; lacks focus or depth; time management is inconsistent	Explanation is unclear or incomplete; difficulty conveying ideas in English; poor time control; limited understanding shown	Unable to explain the design, logic, or code structure; pitch is largely not delivered in English or significantly exceeds/falls short of time
Organization, Terminology, and Oral Delivery (Pitch Delivery)	Pitch is well-structured (problem → solution → how it works → value); uses appropriate computing terminology accurately and consistently; delivery is confident, engaging, and well-paced for 3 minutes	Pitch is organized with minor issues in flow, fluency, or terminology; delivery is clear and mostly well-paced	Pitch is understandable but poorly organized; limited or inconsistent use of computing terminology; pacing is uneven	Pitch is disorganized; frequent terminology errors; delivery shows difficulty communicating ideas in English; pacing is poor	Pitch is unclear, disorganized, and ineffective; lacks appropriate computing terminology and time awareness